

MODULE 4 · NETWORKING

Services

Stable front doors for ephemeral Pods

Pod IPs come and go — clients need one that stays

Every time a Pod is rescheduled it gets a **new IP**. Nothing can hard-code a Pod IP and keep working.

A **Service** is a stable virtual IP and DNS name that load-balances across a changing set of Pods, chosen by **label selector**.

BY THE END YOU CAN

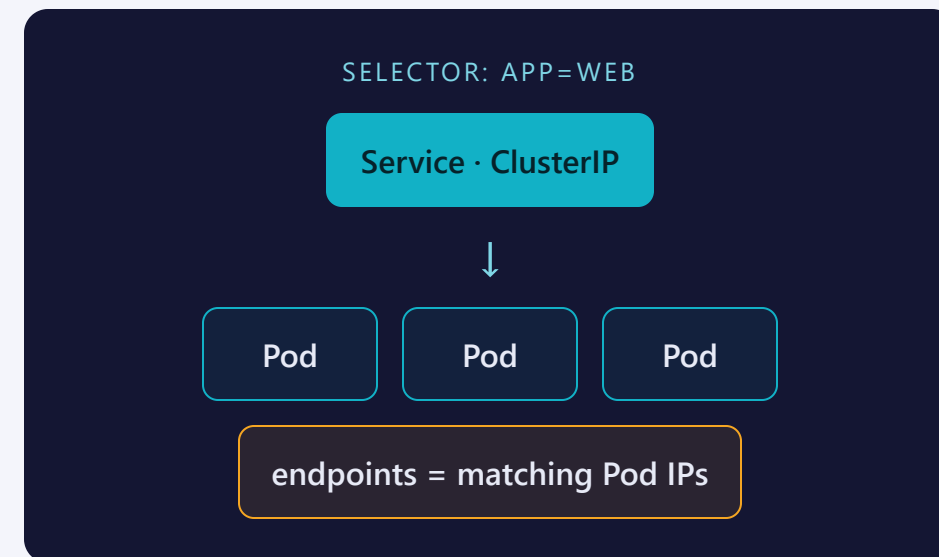
- Explain selectors & endpoints
- Pick ClusterIP / NodePort / LoadBalancer
- Reach a Service inside & outside
- Spot a "no endpoints" mistake

CONCEPT

One virtual IP, many Pods behind it

A Service watches for Pods whose labels match its selector and keeps their IPs as its **endpoints**.

Traffic to the Service IP (or its cluster DNS name) is spread across those endpoints — Pods can come and go underneath.



THE THREE TYPES

Reach grows outward

TYPE	REACHABLE FROM	USE CASE
ClusterIP default	Inside the cluster only	Internal service-to-service
NodePort	Any node IP on a port (30000–32767)	Simple external access, on-prem, dev
LoadBalancer	An external IP — if a load-balancer controller exists	Front a single service with one external IP

EACH BUILDS ON THE LAST

NodePort is a ClusterIP plus a node port; LoadBalancer is a NodePort plus an external front end.

Selector + ports = a Service

```
apiVersion: v1
kind: Service
metadata:
  name: web-clusterip
  namespace: training
spec:
  type: ClusterIP
  selector:
    app: web      # which Pods
  ports:
    - port: 80    # Service port — what clients hit
      targetPort: 80 # container port
      protocol: TCP
```

- **selector** — the labels a Pod must carry to receive traffic
- **port** — what clients hit on the Service
- **targetPort** — the port the container listens on

WATCH

port ≠ targetPort. Mixing them up sends traffic to a closed port.

Expose fast, or declare it

IMPERATIVE

```
kubectl expose deploy web \  
--port=80 --target-port=8080
```

DECLARATIVE

```
kubectl apply -f service.yaml
```

THE BRIDGE TRICK

```
$ kubectl expose deploy web --port=80 --target-port=8080 \  
--dry-run=client -o yaml > service.yaml
```

SEE IT

EndpointSlice proves the wiring

A healthy Service lists one endpoint per matching Pod. **Empty endpoints** = selector matches nothing → connection refused.

EndpointSlice is the modern object backing this — the older Endpoints (v1) API is being phased out, though it's still populated for legacy tooling.

```
$ kubectl get svc,endslices
```

NAME	TYPE	CLUSTER-IP	PORT
web	ClusterIP	10.96.0.10	80

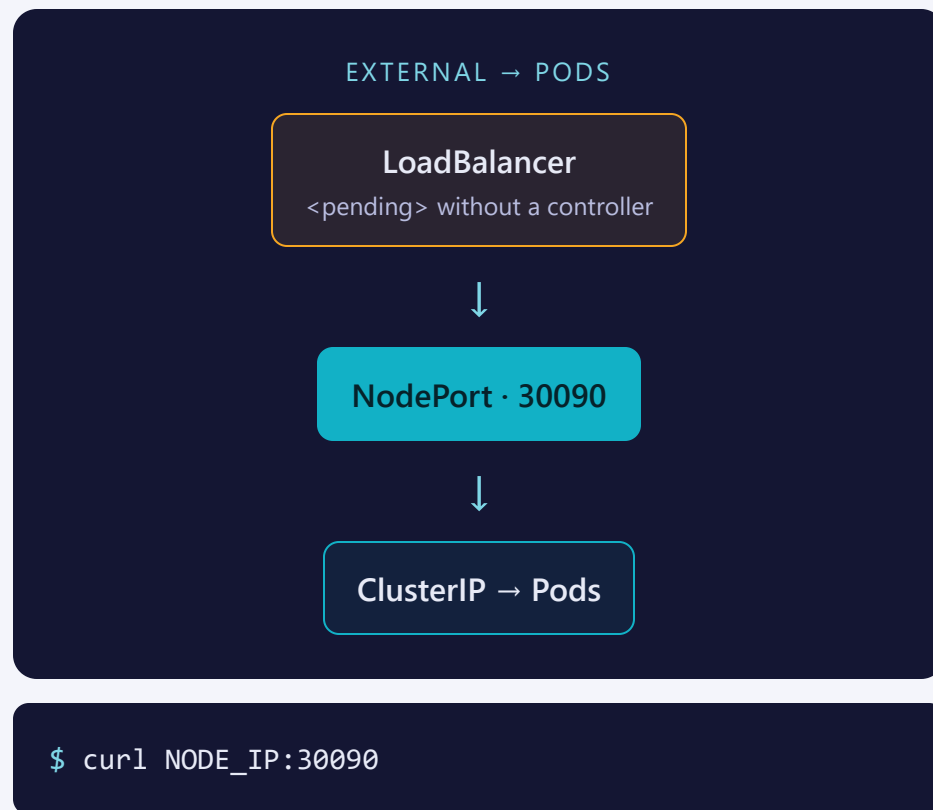
NAME	ADDRESSTYPE	PORTS	ENDPOINTS
web-x7k2p	IPv4	8080	10.1.1.4,10.1.1.5

On a bare cluster, NodePort is the door

A type: LoadBalancer Service asks an **external load-balancer controller** to provision an IP. If the cluster has no such controller, EXTERNAL-IP stays <pending> forever — that's expected, not a failure.

Underneath, LoadBalancer **is** a NodePort with a front end. Here's the NodePort Service actually doing the work:

```
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
  namespace: training
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30090 # 30000-32767 range
      protocol: TCP
```



Where people trip

- **Selector typo.** No Pods match → zero endpoints → "connection refused".
- **port vs targetPort.** Service port is fine; targetPort must equal the container's real port.
- **Expecting an external IP** on a cluster with no load-balancer controller — it stays <pending>.
- **Wrong namespace.** DNS is `svc.ns.svc.cluster.local` — cross-namespace needs the full name.

Commands to keep at hand

```
$ kubectl expose deploy NAME --port=80      # create a ClusterIP
$ kubectl get svc                            # type, cluster IP, ports
$ kubectl get endpointslices -l kubernetes.io/service-name=NAME # are any Pods wired in?
$ kubectl describe svc NAME                  # selector + endpoints
$ curl CLUSTER_IP / curl NODE_IP:NODEPORT # test reachability
```

Tip: when a Service "doesn't work", check `get endpointslices` first — empty means a selector problem, not a network one.

RECAP

Services in one breath

- Service = stable IP + DNS selecting Pods by label
- EndpointSlice = the matching Pod IPs (legacy Endpoints API is being phased out)
- ClusterIP (internal) → NodePort (node) → LoadBalancer (external)
- No controller → LoadBalancer stays <pending>; use NodePort

HANDS-ON

Now run Lab 4.1
labs/4.1/

Next: [4.2 — CoreDNS & Service Discovery](#)